COMSATS University Islamabad

Attock Campus



Lab Midterm Examination

Assignment Submission Portal with Automated Testing

Course	CSC418 – DevOps
Submitted By	Haroon Ijaz
Registration No.	FA23-BSE-002
Submitted To	Sir Muhammad Kamran
Date	30 October 2026

Table of Contents

Submission Details **2**

Part 1 – Overview and Preparation **3**

1.1 What will be built 4

1.2 Tools required 4

1.3 Project structure 4

Part 2 – Develop the Submission Portal **4**

2.1 package.json 5

2.2 Create server.js 5

2.3 Run and try the portal 12

Part 3 – Maintain the Project on GitHub **14**

3.1 .gitignore and README.md 15

3.2 GitHub Repository 16

3.3 Push the project 17

Part 4 – Run Automated Tests on Every Push **19**

4.1 Write the test cases 20

4.2 Run the tests locally 21

4.3 Create the CI pipeline 21

4.4 Push and check the pipeline 22

Part 5 – Send the Report to Localhost and Save JSON **23**

5.1 Create the receiver 24

5.2 Create the report builder 24

5.3 Start the receiver 26

5.4 Start the tunnel 26

5.5 Add the repository secrets 27

5.6 Add the report job to the pipeline 27

5.7 Push and verify 28

Part 6 – Demonstrate the Complete Workflow **29**

6.1 Run 1 – passing run 30

6.2 Run 2 – introduce a defect and fail 30

6.3 Run 3 – fix the defect 32

Part 7 – Bonus Tasks **33**

7.1 Conditional deployment to GitHub Pages 34

Enable Pages 34

Add the deploy job 34

7.2 Email report 36

Add secrets 36

Create scripts/send-email.js 37

Add the email step 37

Submission Details

Table 1: *Summary of the submission*

Item	Details
GitHub repository URL	https://github.com/HaroonIjaz123/HaroonIjaz-FA23-BSE-002-DEVOPS-LAB-MID
Technology	Node.js 22, React 19 with Vite (front end), Express 5 (API), node:test (tests), GitHub Actions (CI)
Local report receiver	`receiver.js` Express HTTP server on port 5050, exposed to GitHub through an ngrok tunnel
Report file	`test-report.json` saved automatically on the local computer after every push
Conditional deployment	GitHub Pages. Deployed URL: https://haroonijaz123.github.io/HaroonIjaz-FA23-BSE-002-DEVOPS-LAB-MID/
Email report	harooonijaz123@gmail.com
Date of submission	27 October 2026

Part 1 – Overview and Preparation

1.1 What will be built

Students enter their name, registration number, assignment title and a submission link. The portal validates the input, saves valid submissions and lists them. Every push to GitHub starts a pipeline that runs the automated tests, and the results are sent to a small receiver running on my own computer, which writes them to `test-report.json`.

1.2 Tools required

Table 2: *Tools required and the command that checks each one*

Tool	Purpose	Check command
Node.js 22 or newer (with npm)	Application, tests, receiver, build	`node --version`
Git	Version control	`git --version`
GitHub account	Repository and Actions	Sign in at github.com
ngrok (free account)	Tunnel from GitHub to localhost	`ngrok version`

1.3 Project structure

Project folder

```
HaroonIjaz(FA23-BSE-002)-DEVOPS-LAB-MID/
├── .github/
│   └── workflows/
│       └── ci.yml              CI pipeline (test, report, deploy)
├── scripts/
│   ├── build-report.js         builds the JSON test report
│   ├── send-report.js          posts the report to the receiver
│   └── send-email.js           e-mails the report (bonus)
├── src/
│   ├── main.jsx                React portal page
│   ├── styles.css              portal styling
│   └── validation.js           shared validation rules
├── test/
│   └── server.test.js          automated tests (node:test)
├── .gitignore
├── index.html                  Vite entry page
├── package.json                dependencies and npm scripts
├── README.md
├── receiver.js                 local report receiver (port 5050)
├── server.js                   Express submission API
└── vite.config.js              Vite settings
```

Figure 1: *Project folder HaroonIjaz(FA23-BSE-002)-DEVOPS-LAB-MID*

Part 2 – Develop the Submission Portal

The portal is a React application built with Vite, served by an Express API. The validation rules are written once in `src/validation.js` and are used by both the React page and the API. The data-file path is a parameter of `loadSubmissions()`, `saveSubmission()` and `createApp()`, so the tests can use a temporary file instead of the real data.

2.1 package.json

`package.json` lists the dependencies and the npm scripts used in the rest of this report (`start`, `build`, `test`, `receiver`, `build-report`, `send-report` and `send-email`).

package.json

```
{
  "name": "assignment-submission-portal",
  "version": "1.0.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite",
    "start": "node server.js",
    "receiver": "node receiver.js",
    "build": "vite build",
    "test": "node --test --test-reporter=spec test/server.test.js",
    "build-report": "node scripts/build-report.js",
    "send-report": "node scripts/send-report.js",
    "send-email": "node scripts/send-email.js"
  },
  "dependencies": {
    "express": "^5.1.0",
    "nodemailer": "^10.3.1",
    "react": "^19.1.0",
    "react-dom": "^19.1.0"
  },
  "devDependencies": {
    "vite": "^7.1.7"
  }
}
```

2.2 Create server.js

The Express server exposes `GET /api/submissions` and `POST /api/submissions`, validates every request, stores accepted records in `submissions.json` and serves the built React site from the `dist` folder.

server.js

```
import express from 'express';
import { readFile, writeFile } from 'node:fs/promises';
import { existsSync } from 'node:fs';
import path from 'node:path';
import { fileURLToPath } from 'node:url';
import { validateSubmission } from './src/validation.js';

const here = path.dirname(fileURLToPath(import.meta.url));
export const dataFile = process.env.SUBMISSIONS_FILE || path.join(here, 'submissions.json');

export async function loadSubmissions(file = dataFile) {
  try {
    return JSON.parse(await readFile(file, 'utf8'));
  } catch (error) {
    if (error.code === 'ENOENT') return [];
    throw error;
  }
}
```

```
export async function saveSubmission(file, record) {
  const submissions = await loadSubmissions(file);
  submissions.push(record);
  await writeFile(file, `${JSON.stringify(submissions, null, 2)}\n`, 'utf8');
  return submissions;
}

export function createApp(file = dataFile) {
  const app = express();
  app.use(express.json());
  const builtSite = path.join(here, 'dist');
  if (existsSync(builtSite)) app.use(express.static(builtSite));

  app.get('/api/submissions', async (_req, res, next) => {
    try { res.json(await loadSubmissions(file)); } catch (error) { next(error); }
  });

  app.post('/api/submissions', async (req, res, next) => {
    const errors = validateSubmission(req.body ?? {});
    if (errors.length) return res.status(400).json({ errors });

    try {
      const record = Object.fromEntries(['name', 'reg_no', 'title', 'url']
        .map((field) => [field, req.body[field].trim()]));
      const submissions = await saveSubmission(file, record);
      return res.status(201).json({ submission: record, submissions });
    } catch (error) { return next(error); }
  });

  app.use((error, _req, res, _next) => {
    console.error(error);
    res.status(500).json({ error: 'Unable to process the request.' });
  });
  return app;
}

if (process.argv[1] === fileURLToPath(import.meta.url)) {
  const port = Number(process.env.PORT || 5000);
  const server = createApp().listen(port, () => console.log(`Portal listening on
http://localhost:${port}`));
  server.on('error', (error) => { console.error('Portal failed to start:', error.message);
process.exitCode = 1; });
}
```

Validation rules. The same functions run in the browser (before the request is sent) and on the server (before the record is saved).

src/validation.js

```
export function isValidUrl(value) {
  const trimmed = (value ?? '').trim();
  if (!trimmed || /\s/.test(trimmed) || !/^https?:\/\/[^/?#]+/i.test(trimmed)) return false;

  try {
    const parsed = new URL(trimmed);
    return ['http:', 'https:'].includes(parsed.protocol) && Boolean(parsed.host);
  } catch {
    return false;
  }
}

export function validateSubmission(form) {
  const errors = [];
  if (!(form.name ?? '').trim()) errors.push('Name is required.');
  if (!(form.reg_no ?? '').trim()) errors.push('Registration number is required.');
  if (!(form.title ?? '').trim()) errors.push('Assignment title is required.');
  const url = (form.url ?? '').trim();
  if (!url) errors.push('Submission URL is required.');
  else if (!isValidUrl(url)) errors.push('Submission URL is not a valid http(s) URL.');
  return errors;
}
```

React portal page. `main.jsx` renders the form and the Saved Submissions table. If the API is not available (for example on GitHub Pages), the page falls back to the browser's local storage.

src/main.jsx

```
import React, { useEffect, useState } from 'react';
import { createRoot } from 'react-dom/client';
import { validateSubmission } from './validation.js';
import './styles.css';

const emptyForm = { name: '', reg_no: '', title: '', url: '' };

function App() {
  const [form, setForm] = useState(emptyForm);
  const [submissions, setSubmissions] = useState([]);
  const [errors, setErrors] = useState([]);
  const [message, setMessage] = useState('');
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [apiAvailable, setApiAvailable] = useState(true);

  useEffect(() => {
    fetch('/api/submissions')
      .then((response) => { if (!response.ok) throw new Error(); return response.json(); })
      .then(setSubmissions)
      .catch(() => {
        setApiAvailable(false);
        try { setSubmissions(JSON.parse(localStorage.getItem('submissions') || '[]')); }
        catch { setSubmissions([]); }
      })
      .finally(() => setLoading(false));
  }, []);

  function update(event) {
    setForm((current) => ({ ...current, [event.target.name]: event.target.value }));
  }

  async function submit(event) {
    event.preventDefault();
    setMessage('');
    const validationErrors = validateSubmission(form);
    setErrors(validationErrors);
    if (validationErrors.length) return;

    setSaving(true);
    try {
      if (!apiAvailable) {
        const next = [...submissions, Object.fromEntries(Object.entries(form).map(([key, value]) => [key,
value.trim()]))];
        localStorage.setItem('submissions', JSON.stringify(next));
        setSubmissions(next);
        setForm(emptyForm);
        setErrors([]);
        setMessage('Submission saved in this browser.');
        return;
      }
      const response = await fetch('/api/submissions', {
        method: 'POST', headers: { 'Content-Type': 'application/json' }, body: JSON.stringify(form),
      });
      const result = await response.json();
      if (!response.ok) { setErrors(result.errors ?? [result.error ?? 'Submission failed.']); return; }
      setSubmissions(result.submissions);
      setForm(emptyForm);
      setErrors([]);
      setMessage('Submission saved.');
    } catch {
      setMessage('Could not save your submission. Please try again.');
    } finally { setSaving(false); }
  }

  return <>
    <header><h1>Assignment Submission Portal</h1><p>CSC418 – DevOps · COMSATS University Islamabad, Attock
Campus</p></header>
```

```
    <main>
      <section className="card">
        <h2>Submit an assignment</h2>
        {errors.length > 0 && <div className="errors" role="alert">{errors.map((error) => <div
className="error" key={error}>{error}</div>)}</div>}
        {message && <p className="status" role="status">{message}</p>}
        <form onSubmit={submit} noValidate>
          <div className="grid">
            <Field label="Name" name="name" placeholder="e.g. Haroon Ijaz" value={form.name}
onChange={update} />
            <Field label="Registration Number" name="reg_no" placeholder="e.g. FA23-BSE-002"
value={form.reg_no} onChange={update} />
            <Field className="full" label="Assignment Title" name="title" placeholder="e.g. Lab Assignment
2" value={form.title} onChange={update} />
            <Field className="full" label="Submission URL" name="url"
placeholder="https://github.com/username/repository" value={form.url} onChange={update} />
          </div>
          <button type="submit" disabled={saving}>{saving ? 'Saving…' : 'Submit'}</button>
        </form>
      </section>
      <section className="card">
        <h2>Saved Submissions <span className="badge">{submissions.length}</span></h2>
        <div className="table-wrap"><table id="submissions">
          <thead><tr><th>Name</th><th>Reg. No</th><th>Assignment</th><th>URL</th></tr></thead>
          <tbody>{loading ? <tr><td className="empty" colSpan="4">Loading submissions…</td></tr> :
submissions.length ? submissions.map((item, index) => <tr key={`${item.reg_no}-${index}`}>
            <td data-label="Name">{item.name}</td><td data-label="Reg. No">{item.reg_no}</td><td data-
label="Assignment">{item.title}</td>
            <td data-label="URL"><a href={item.url} target="_blank" rel="noopener
noreferrer">{item.url}</a></td>
          </tr>) : <tr><td className="empty" colSpan="4">No submissions yet. Be the first to
submit.</td></tr>}</tbody>
        </table></div>
      </section>
    </main>
    <footer>Haroon Ijaz · FA23-BSE-002 · CSC418 Lab Midterm</footer>
  </>;
}

function Field({ label, name, className = '', ...props }) {
  return <div className={className}><label htmlFor={name}>{label}</label><input id={name} name={name}
{...props} /></div>;
}

createRoot(document.getElementById('root')).render(<App />);
```

Page template, build settings and styling.

index.html

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#1f3864" />
    <title>Assignment Submission Portal</title>
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

vite.config.js

```
import { defineConfig } from 'vite';

export default defineConfig({
  base: './',
  server: { proxy: { '/api': 'http://localhost:5000' } },
});
```

src/styles.css (formatted for readability)

```
:root {
  font-family:"Segoe UI",system-ui,-apple-system,Roboto,Arial,sans-serif;
  color:#1d2433;
  background:#f2f5fb;
  font-synthesis:none;
  text-rendering:optimizeLegibility;
  line-height:1.5;
  font-weight:400;
  --navy:#1f3864;
  --blue:#2e5597;
  --line:#e3e8f2;
  --muted:#667085;
}

* {
  box-sizing:border-box;
}

body {
  margin:0;
}

header {
  background:linear-gradient(135deg,var(--navy),var(--blue));
  color:#fff;
  padding:36px 16px 70px;
  text-align:center;
}

header h1 {
  margin:0 0 6px;
  font-size:clamp(1.5rem,4vw,2.2rem);
}

header p {
  margin:0;
  opacity:.85;
  font-size:.95rem;
}

main {
  max-width:980px;
  margin:-44px auto 40px;
  padding:0 16px;
}

.card {
  background:#fff;
  border-radius:14px;
  padding:24px;
  box-shadow:0 6px 24px #1f38641a;
  margin-bottom:24px;
}

.card h2 {
  margin:0 0 16px;
  font-size:1.2rem;
  color:var(--navy);
}

.grid {
  display:grid;
  grid-template-columns:1fr 1fr;
  gap:16px;
}

.full {
  grid-column:1/-1;
}

label {
```

```
  display:block;
  margin-bottom:6px;
  font-weight:600;
  font-size:.9rem;
}

input {
  width:100%;
  padding:11px 12px;
  font-size:1rem;
  border:1px solid #cfd7e6;
  border-radius:8px;
  background:#fbfcff;
}

input:focus {
  outline:none;
  border-color:var(--blue);
  background:#fff;
  box-shadow:0 0 0 3px #2e55972e;
}

button {
  margin-top:20px;
  width:100%;
  padding:13px 20px;
  font-size:1rem;
  font-weight:600;
  color:#fff;
  border:0;
  border-radius:8px;
  cursor:pointer;
  background:linear-gradient(135deg,var(--blue),var(--navy));
}

button:disabled {
  opacity:.7;
  cursor:wait;
}

.errors {
  margin-bottom:16px;
}

.error {
  background:#fdecea;
  color:#b71c1c;
  padding:10px 14px;
  border-left:4px solid #b71c1c;
  border-radius:6px;
  margin-bottom:8px;
  font-size:.93rem;
}

.status {
  margin:0 0 14px;
  color:#1b7f4b;
}

.badge {
  display:inline-block;
  margin-left:8px;
  padding:1px 10px;
  font-size:.8rem;
  color:#fff;
  background:var(--blue);
  border-radius:999px;
  vertical-align:middle;
}

.table-wrap {
  overflow-x:auto;
}

table {
  border-collapse:collapse;
```

```
  width:100%;
}

th {
  background:var(--navy);
  color:#fff;
  text-align:left;
  padding:11px 12px;
  font-size:.88rem;
  white-space:nowrap;
}

th:first-child {
  border-radius:8px 0 0;
}

th:last-child {
  border-radius:0 8px 0 0;
}

td {
  padding:11px 12px;
  border-bottom:1px solid var(--line);
  word-break:break-word;
}

tbody tr:nth-child(even) {
  background:#f8faff;
}

tbody tr:hover {
  background:#eef3fc;
}

td[data-label="Reg. No"] {
  white-space:nowrap;
}

td a {
  color:var(--blue);
  text-decoration:none;
}

td a:hover {
  text-decoration:underline;
}

.empty {
  text-align:center;
  color:var(--muted);
  padding:28px 8px;
}

footer {
  text-align:center;
  color:var(--muted);
  font-size:.82rem;
  padding-bottom:28px;
}

@media(max-width:640px) {
  header {
    padding:28px 16px 60px;
  }
  .card {
    padding:18px;
  }
  .grid {
    grid-template-columns:1fr;
  }
  thead {
    display:none;
  }
  table,tbody,tr,td {
    display:block;
    width:100%;
```

```
  }
  tbody tr {
    border:1px solid var(--line);
    border-radius:10px;
    margin-bottom:12px;
    padding:6px 0;
    background:#fff!important;
  }
  td {
    border:0;
    padding:6px 14px;
  }
  td[data-label="Reg. No"] {
    white-space:normal;
  }
  td::before {
    content:attr(data-label);
    display:block;
    font-size:.75rem;
    font-weight:700;
    text-transform:uppercase;
    color:var(--muted);
  }
}
```

2.3 Run and try the portal

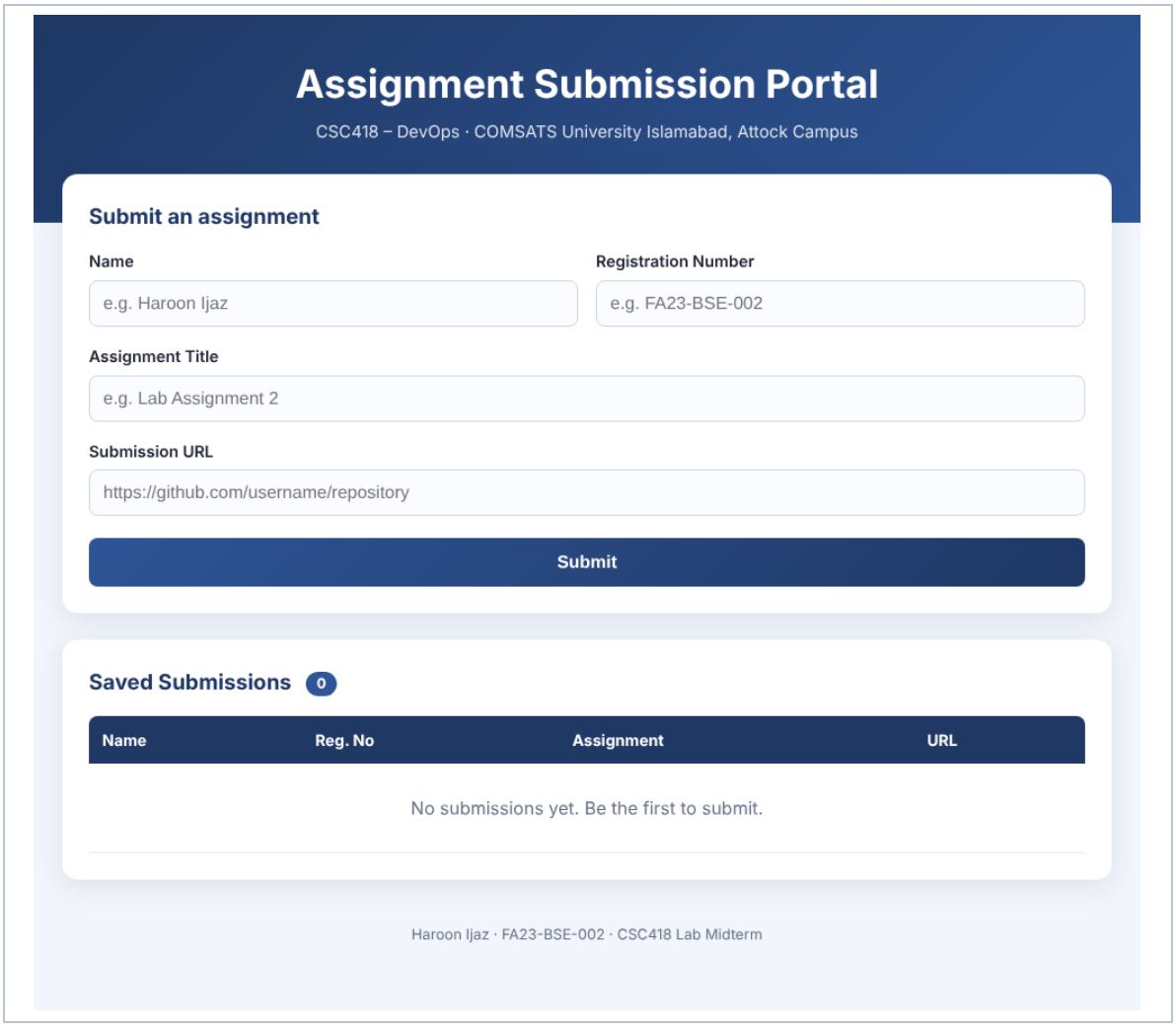


Figure 2: *Portal home page with the empty form*

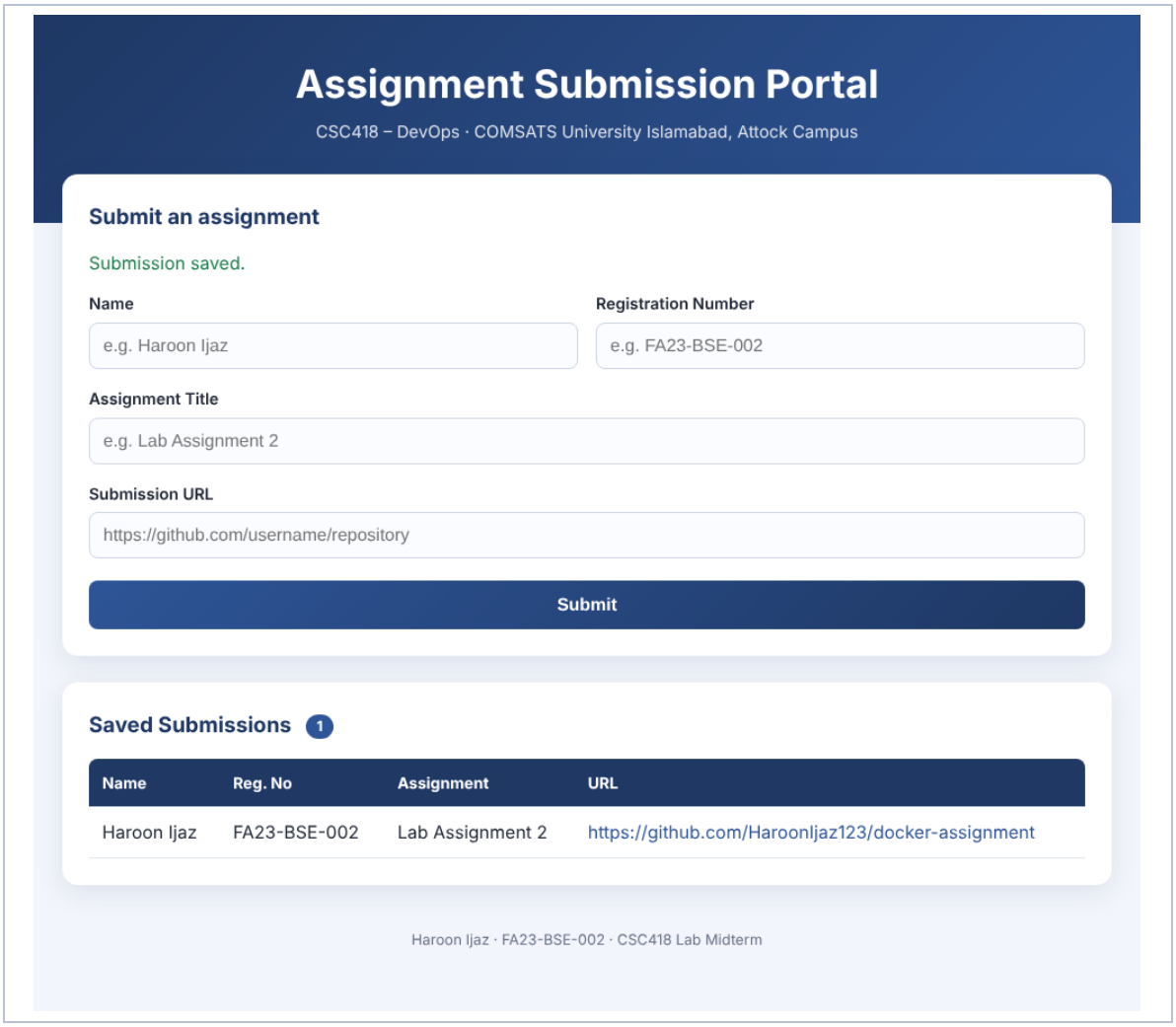


Figure 3: *Valid submission saved and shown in the Saved Submissions table*

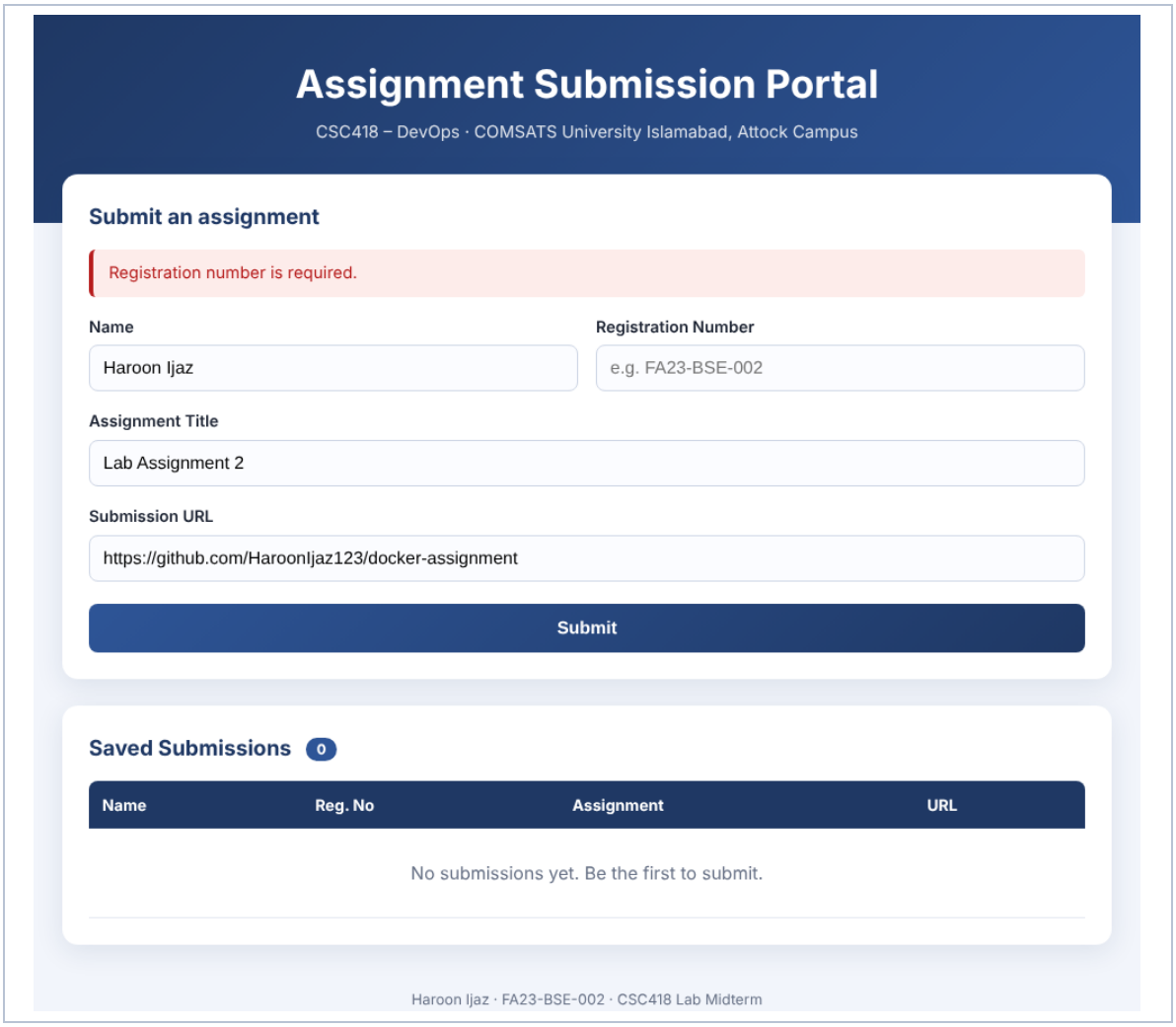


Figure 4: *Validation message for the missing registration number*

Part 3 – Maintain the Project on GitHub

3.1 .gitignore and README.md

.gitignore

```
__pycache__/
.pytest_cache/
venv/
node_modules/
dist/
submissions.json
test-report.json
test-output.txt
report-delivery-error.txt
```

README.md

````
# Assignment Submission Portal

CSC418 DevOps Lab Midterm | Haroon Ijaz (FA23-BSE-002)

This project uses React for the portal and Node.js for the submission API, CI report sender, email sender,
and local report receiver. The API stores submissions in `submissions.json`. GitHub Pages is static, so
its build stores submissions in the visitor's browser.

## Requirements

- Node.js 22 or newer and npm
- ngrok installed and connected to an ngrok account for GitHub Actions to reach your local report receiver

## Run the portal locally

```bash
npm ci
npm start
```

Open http://localhost:5000. The API saves submissions in `submissions.json`. Set `PORT` to change the app
port or `SUBMISSIONS_FILE` to choose another data file.

For frontend development with Vite, run `npm start` in one terminal and `npm run dev` in another, then
open http://localhost:5173.

## Test and build

```bash
npm test
npm run build
```

The existing valid submission, missing registration number, and invalid URL cases remain covered.
Additional tests cover required fields, whitespace, URL variants, trimming, JSON persistence, and CI
report summaries.

## Send GitHub Actions reports to your computer with ngrok

GitHub-hosted Actions cannot reach `localhost` on your computer directly. Keep the report receiver and
ngrok tunnel running while a GitHub Actions run is sending its report.

1. In your ngrok account, get an authtoken and configure the ngrok agent:

   ```bash
   ngrok config add-authtoken "YOUR_NGROK_AUTHTOKEN"
   ```
````

2. Start the report receiver in a terminal. Use the same long random token in the receiver and GitHub secret:

   ```bash
   REPORT_TOKEN="your-long-random-report-token" npm run receiver
   ```

   The receiver listens on port 5050 and saves reports as `test-report.json` in the project directory. You can set `RECEIVER_PORT` and `REPORT_FILE` to change those defaults.

3. In a second terminal, expose the receiver port:

   ```bash
   ngrok http 5050
   ```

   Copy the HTTPS forwarding address printed by ngrok and append `/report`, for example `https://example.ngrok-free.app/report`.

4. In the GitHub repository, open **Settings → Secrets and variables → Actions** and add:

Secret	Value
`RECEIVER_URL`	Your ngrok HTTPS URL ending in `/report`
`REPORT_TOKEN`	The exact token used to start the local receiver
`SMTP_USERNAME`	Sender email address (optional)
`SMTP_PASSWORD`	Sender app password (optional)
`EMAIL_TO`	Report recipient (optional)

Push to GitHub while both local processes are running. Actions runs the tests and React build, creates a JSON report, posts it through the ngrok tunnel, and stores the report as an Actions artifact. If ngrok gives you a new URL, update `RECEIVER_URL`. If the receiver or tunnel is offline, report delivery fails and the report remains available as an artifact. Email is skipped unless SMTP credentials are set.

To send a report manually from the project folder, first create it, then post it:

```bash
npm test > test-output.txt
npm run build-report
npm run send-report
```

GitHub Pages

The workflow deploys the static React build on pushes to `main`. Enable GitHub Pages using the `github-pages` environment if the repository has not used Pages before. The Pages version uses browser storage; the Node API is not hosted by GitHub Pages.

Project layout

```text
src/main.jsx               React portal
src/validation.js          Submission validation
src/styles.css             Portal styling
server.js                  Express submission API and JSON persistence
receiver.js                Node report receiver
scripts/build-report.js    Creates detailed CI JSON report
scripts/send-report.js     Posts the report to the ngrok receiver
scripts/send-email.js      Optional SMTP email with JSON attachment
test/server.test.js        Node test suite
.github/workflows/ci.yml  Tests, report delivery, email, and Pages deployment
```

3.2 GitHub Repository

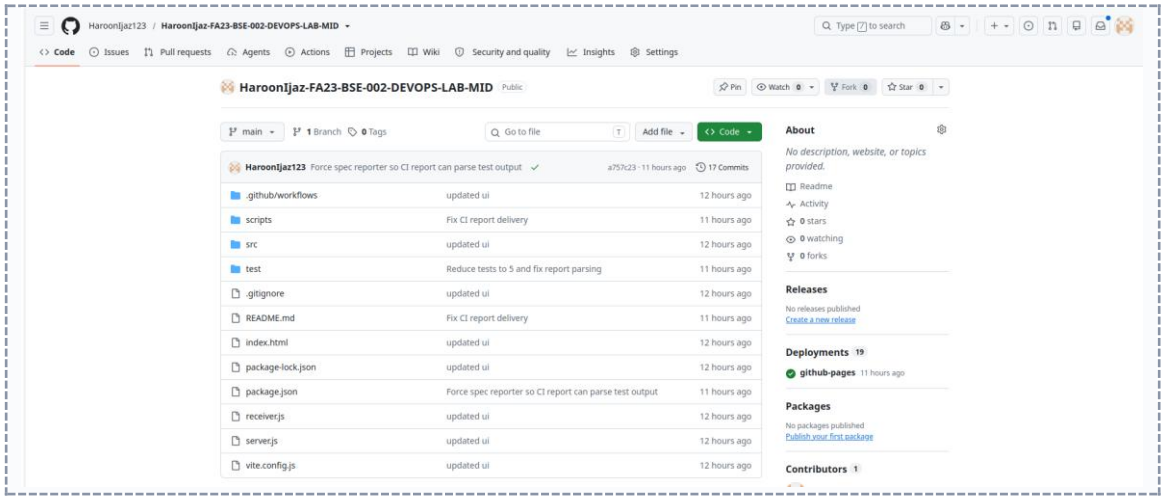


Figure 5: *GitHub Repository page*

3.3 Push the project

Terminal

```
git add .
git commit -m "Add assignment submission portal"
git push
```

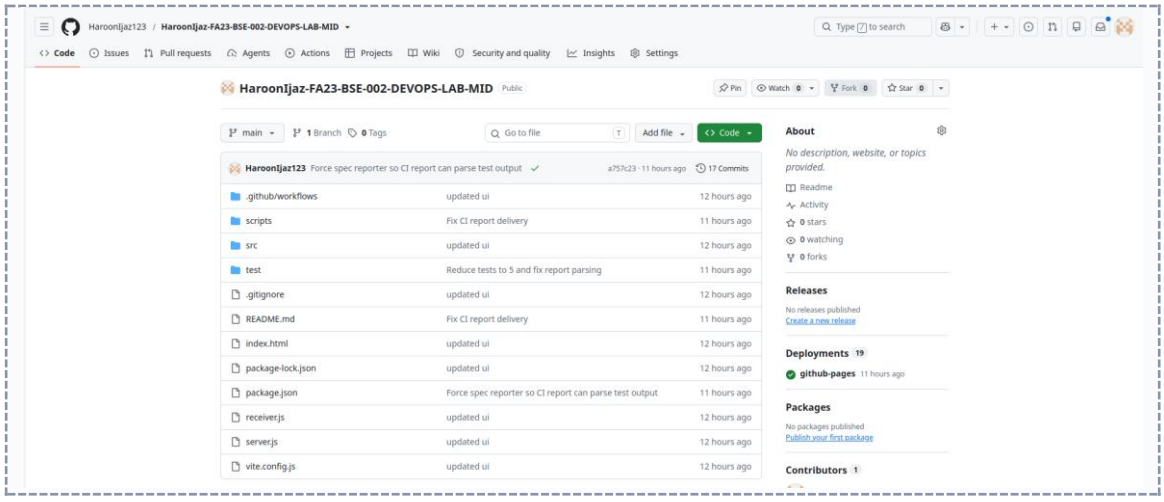


Figure 6: *GitHub repository page showing the pushed files*

README

Assignment Submission Portal

CSC418 DevOps Lab Midterm | Haroon Ijaz (FA23-BSE-002)

This project uses React for the portal and Node.js for the submission API, CI report sender, email sender, and local report receiver. The API stores submissions in `submissions.json`. GitHub Pages is static, so its build stores submissions in the visitor's browser.

Requirements

- Node.js 22 or newer and npm
- ngrok installed and connected to an ngrok account for GitHub Actions to reach your local report receiver

Run the portal locally

```
npm ci
npm start
```

Open http://localhost:5000. The API saves submissions in `submissions.json`. Set `PORT` to change the app port or `SUBMISSIONS_FILE` to choose another data file.

For frontend development with Vite, run `npm start` in one terminal and `npm run dev` in another, then open http://localhost:5173.

Test and build

```
npm test
npm run build
```

The existing valid submission, missing registration number, and invalid URL cases remain covered. Additional tests cover required fields, whitespace, URL variants, trimming, JSON persistence, and CI report summaries.

Send GitHub Actions reports to your computer with ngrok

GitHub-hosted Actions cannot reach `localhost` on your computer directly. Keep the report receiver and ngrok tunnel running while a GitHub Actions run is sending its report.

1. In your ngrok account, get an authtoken and configure the ngrok agent:

   ```
   ngrok config add-authtoken "YOUR_NGROK_AUTHTOKEN"
   ```

2. Start the report receiver in a terminal. Use the same long random token in the receiver and GitHub secret:

   ```
   REPORT_TOKEN="your-long-random-report-token" npm run receiver
   ```

   The receiver listens on port 5050 and saves reports as `test-report.json` in the project directory. You can set `RECEIVER_PORT` and `REPORT_FILE` to change those defaults.

3. In a second terminal, expose the receiver port:

   ```
   ngrok http 5050
   ```

   Copy the HTTPS forwarding address printed by ngrok and append `/report`, for example `https://example.ngrok-free.app/report`.

3. In a second terminal, expose the receiver port:

```
ngrok http 5050
```

Copy the HTTPS forwarding address printed by ngrok and append `/report`, for example `https://example.ngrok-free.app/report`.

4. In the GitHub repository, open **Settings → Secrets and variables → Actions** and add:

Secret	Value
`RECEIVER_URL`	Your ngrok HTTPS URL ending in `/report`
`REPORT_TOKEN`	The exact token used to start the local receiver
`SMTP_USERNAME`	Sender email address (optional)
`SMTP_PASSWORD`	Sender app password (optional)
`EMAIL_TO`	Report recipient (optional)

Push to GitHub while both local processes are running. Actions runs the tests and React build, creates a JSON report, posts it through the ngrok tunnel, and stores the report as an Actions artifact. If ngrok gives you a new URL, update `RECEIVER_URL`. If the receiver or tunnel is offline, report delivery fails and the report remains available as an artifact. Email is skipped unless SMTP credentials are set.

To send a report manually from the project folder, first create it, then post it:

```
npm test > test-output.txt
npm run build-report
npm run send-report
```

GitHub Pages

The workflow deploys the static React build on pushes to `main`. Enable GitHub Pages using the `github-pages` environment if the repository has not used Pages before. The Pages version uses browser storage; the Node API is not hosted by GitHub Pages.

Project layout

```
src/main.jsx               React portal
src/validation.js          Submission validation
src/styles.css             Portal styling
server.js                  Express submission API and JSON persistence
receiver.js                Node report receiver
scripts/build-report.js    Creates detailed CI JSON report
scripts/send-report.js     Posts the report to the ngrok receiver
scripts/send-email.js      Optional SMTP email with JSON attachment
test/server.test.js        Node test suite
.github/workflows/ci.yml  Tests, report delivery, email, and Pages deployment
```

Figure 7: *README.md rendered on the repository home page*

Part 4 – Run Automated Tests on Every Push

4.1 Write the test cases

The tests use the `node:test` runner that is built into Node.js, so no extra test library is needed. Five test cases are defined:

Table 3: *Automated test cases*

Test case	Input	Expected result
`valid submission is saved`	All fields filled, valid URL	No validation errors; submission is saved and appears in the list
`missing registration number is rejected and not saved`	Registration number empty	Message “Registration number is required.” and nothing saved
`invalid URL is rejected and not saved`	URL is not-a-valid-url	Message “Submission URL is not a valid http(s) URL.” and nothing saved
`valid URLs are accepted and bad URLs are rejected`	https URL; ftp URL, missing host, URL with a space, empty text	Only the https URL is accepted
`CI report builder keeps individual pass, fail, and skipped test results`	Sample test-runner output with one passed, one failed and one skipped test	Report lists each test with its result; overall_status = FAILED

test/server.test.js

```
import test from 'node:test';
import assert from 'node:assert/strict';
import os from 'node:os';
import path from 'node:path';
import { mkdtemp, rm } from 'node:fs/promises';
import { loadSubmissions, saveSubmission } from '../server.js';
import { isValidUrl, validateSubmission } from '../src/validation.js';
import { buildReport } from '../scripts/build-report.js';

const valid = {
  name: 'Haroon Ijaz', reg_no: 'FA23-BSE-002', title: 'Lab Assignment 2',
  url: 'https://github.com/haroon/lab2',
};

async function withDataFile(run) {
  const directory = await mkdtemp(path.join(os.tmpdir(), 'submission-portal-'));
  const file = path.join(directory, 'submissions.json');
  try { await run(file); }
  finally { await rm(directory, { recursive: true, force: true }); }
}

test('valid submission is saved', async () => withDataFile(async (file) => {
  assert.deepEqual(validateSubmission(valid), []);
  await saveSubmission(file, valid);
  assert.deepEqual(await loadSubmissions(file), [valid]);
}));

test('missing registration number is rejected and not saved', async () => withDataFile(async (file) => {
  const errors = validateSubmission({ ...valid, reg_no: '' });
  assert.ok(errors.includes('Registration number is required.'));
  assert.deepEqual(await loadSubmissions(file), []);
}));
```

```
test('invalid URL is rejected and not saved', async () => withDataFile(async (file) => {
  const errors = validateSubmission({ ...valid, url: 'not-a-valid-url' });
  assert.ok(errors.includes('Submission URL is not a valid http(s) URL.'));
  assert.deepEqual(await loadSubmissions(file), []);
}));

test('valid URLs are accepted and bad URLs are rejected', () => {
  assert.equal(isValidUrl('https://example.com/path'), true);
  for (const url of ['ftp://example.com/file', 'https:///missing-host', 'https://example.com/a b', '']) {
    assert.equal(isValidUrl(url), false, url);
  }
});

test('CI report builder keeps individual pass, fail, and skipped test results', () => {
  const output = [
    '✓ accepts a valid submission (2.1ms)',
    '✕ rejects malformed URLs (1.2ms)',
    ' - skips optional integration check (0.2ms)',
    'ℹ tests 3', 'ℹ pass 1', 'ℹ fail 1', 'ℹ skipped 1',
  ].join('\n');
  const report = buildReport(output, { GITHUB_REPOSITORY: 'owner/repo', GITHUB_SHA: 'abc123' });
  assert.deepEqual(report.tests.map(({ name, result }) => [name, result]), [
    ['accepts a valid submission', 'passed'],
    ['rejects malformed URLs', 'failed'],
    ['skips optional integration check', 'skipped'],
  ]);
  assert.deepEqual(report.summary, { total: 3, passed: 1, failed: 1, skipped: 1, overall_status:
'FAILED' });
  assert.equal(report.repository, 'owner/repo');
});
```

4.2 Run the tests locally

Terminal

```
npm test

> node --test --test-reporter=spec test/server.test.js

✓ valid submission is saved (9.383801ms)
✓ missing registration number is rejected and not saved (3.944576ms)
✓ invalid URL is rejected and not saved (1.789143ms)
✓ valid URLs are accepted and bad URLs are rejected (0.317009ms)
✓ CI report builder keeps individual pass, fail, and skipped test results (1.315515ms)
ℹ tests 5
ℹ suites 0
ℹ pass 5
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 161.735864
```

Figure 8: *Local npm test output showing 5 passed*

4.3 Create the CI pipeline

The `test` job installs the packages with `npm ci`, runs the tests, builds the React application and keeps `test-output.txt` as an artifact for the report job. The workflow starts on every push to every branch.

.github/workflows/ci.yml (test job)

```
name: CI

on:
  push:
    branches: ["**"]
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - name: Run JavaScript tests
        run: npm test > test-output.txt
      - name: Build React application
        run: npm run build
      - name: Keep test output for report job
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: ci-test-output
          path: test-output.txt
```

4.4 Push and check the pipeline

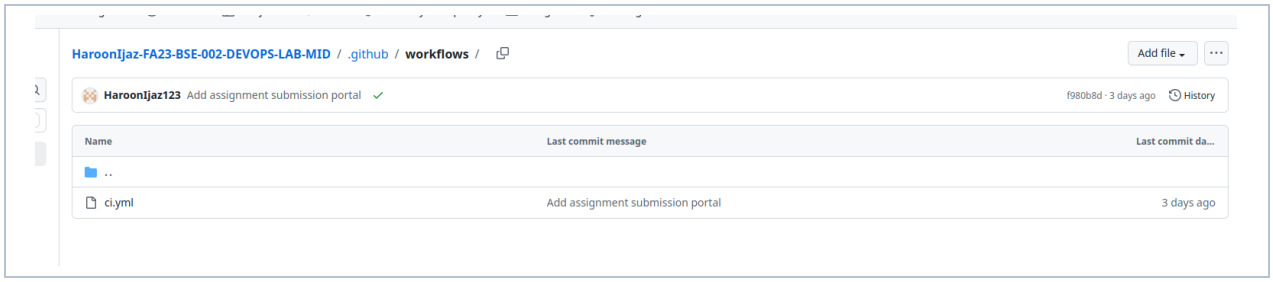


Figure 9: *Workflow file ci.yml in the repository*

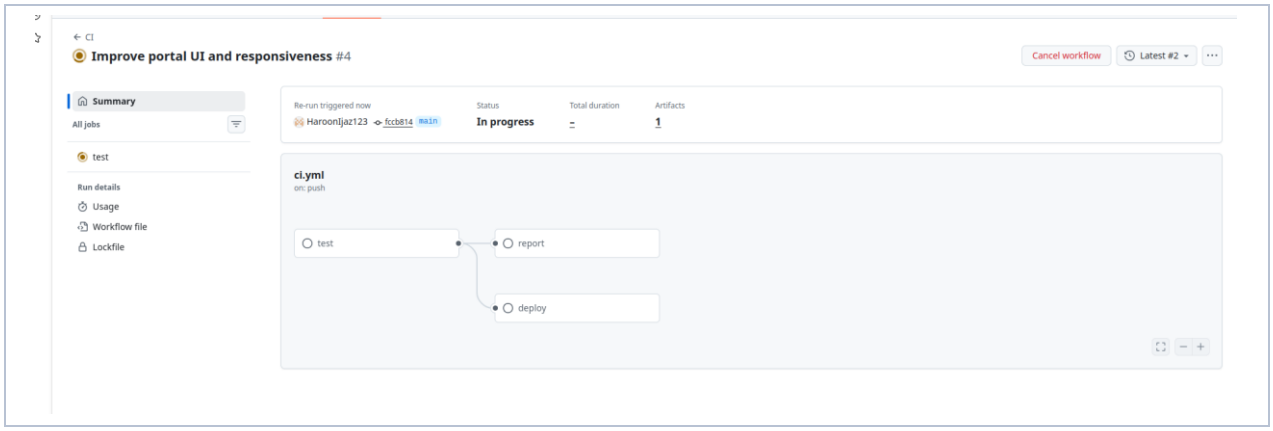


Figure 10: *Actions tab showing the workflow started automatically by the push*

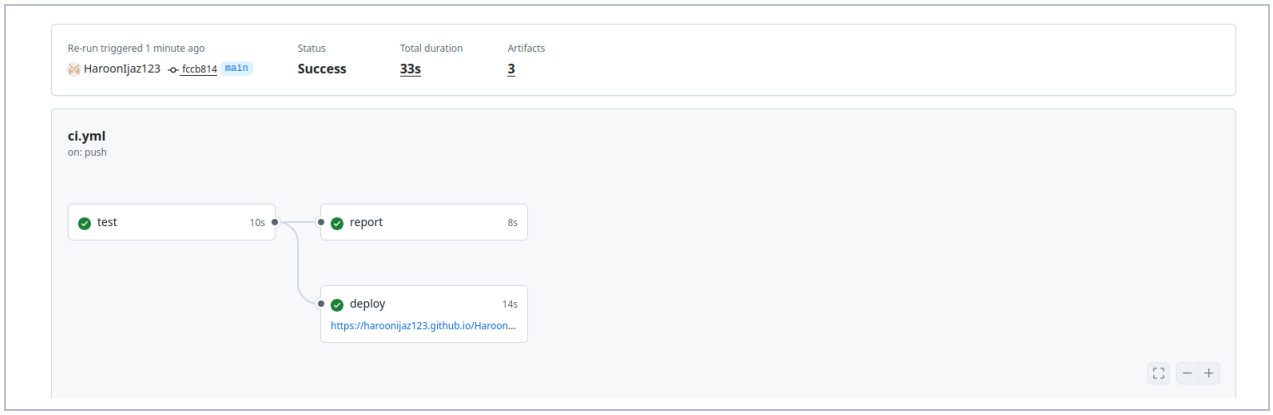


Figure 11: *Actions run finished successfully*

Part 5 – Send the Report to Localhost and Save JSON

A GitHub-hosted runner cannot reach localhost on my computer, because on the runner localhost is the runner itself. The solution is a tunnel: the receiver runs on my computer, ngrok gives it a public HTTPS address, and the pipeline posts the report to that address.

5.1 Create the receiver

`receiver.js` is an Express server. `POST /report` checks the `x-report-token` header, validates the body and saves it as `test-report.json` (written to a temporary file first and then renamed, so a partly written report is never left behind).

receiver.js

```
import express from 'express';
import { rename, writeFile } from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';


const projectDir = path.dirname(fileURLToPath(import.meta.url));
export const reportFile = process.env.REPORT_FILE || path.join(projectDir, 'test-report.json');


export function createReceiver() {
  const app = express();
  app.use(express.json({ limit: '2mb' }));
  app.get('/', (_req, res) => res.json({ message: 'Receiver is running. POST reports to /report' }));
  app.get('/health', (_req, res) => res.json({ status: 'ok' }));
  app.post('/report', async (req, res, next) => {
    const token = process.env.REPORT_TOKEN || '';
    if (token && req.get('x-report-token') !== token) return res.status(401).json({ message: 'Invalid
token' });
    const report = req.body;
    if (!report || typeof report !== 'object' || !report.summary || !Array.isArray(report.tests)) {
      return res.status(400).json({ message: 'Body must be a test report with summary and tests.' });
    }
    try {
      const temporary = `${reportFile}.tmp`;
      await writeFile(temporary, `${JSON.stringify(report, null, 2)}\n`, 'utf8');
      await rename(temporary, reportFile);
      const { summary = {} } = report;
      console.log(`[receiver] saved ${reportFile}: commit=${String(report.commit_sha || '').slice(0, 7)}
status=${summary.overall_status} passed=${summary.passed}/${summary.total}`);
      return res.json({ message: 'Report saved' });
    } catch (error) { return next(error); }
  });
  app.use((error, _req, res, _next) => {
    if (error instanceof SyntaxError && 'body' in error) return res.status(400).json({ message: 'Body is
not valid JSON' });
    console.error(error);
    return res.status(500).json({ message: 'Unable to save report.' });
  });
  return app;
}


if (process.argv[1] && path.resolve(process.argv[1]) === fileURLToPath(import.meta.url)) {
  const port = Number(process.env.RECEIVER_PORT || process.env.PORT || 5050);
  const server = createReceiver().listen(port, '0.0.0.0', () => console.log(`[receiver] listening on
http://localhost:${port}/report`));
  server.on('error', (error) => { console.error('[receiver] failed to start:', error.message);
process.exitCode = 1; });
}
```

5.2 Create the report builder

`scripts/build-report.js` reads the spec-style output of `npm test` (saved in `test-output.txt`), so the results come from real test execution. It lists every test as passed, failed or skipped, adds the repository, branch, commit SHA, run id and timestamp from the GitHub environment variables, computes the totals and overall status, and writes `test-report.json`.

scripts/build-report.js

```
import { readFile, writeFile } from 'node:fs/promises';
import path from 'node:path';
import { fileURLToPath } from 'node:url';

export function buildReport(output, metadata = process.env) {
  const clean = output.replace(/\u001b\[[0-9;]*m/g, '');
  const tests = [];
  for (const line of clean.split(/\r?\n/)) {
    const match = line.match(/^[✓✕ - ]\s+(.+?)(?:\s+\(([\d.]+)ms\))?$/u);
    if (!match) continue;
    const result = line.startsWith('✓') ? 'passed' : line.startsWith('✕') ? 'failed' : 'skipped';
    tests.push({
      name: match[1], result,
      failure_message: result === 'failed' ? 'See the test job log for details.' : null,
    });
  }

  if (!tests.length) {
    const failCount = Number(clean.match(/^ℹ fail (\d+)$/m)?.[1] || 0);
    tests.push({
      name: 'test runner', result: 'failed',
      failure_message: failCount ? 'See the test job log for details.' : 'No individual test results were
found in test-output.txt.',
    });
  }
  const passed = tests.filter((item) => item.result === 'passed').length;
  const failed = tests.filter((item) => item.result === 'failed').length;
  const skipped = tests.filter((item) => item.result === 'skipped').length;
  return {
    repository: metadata.GITHUB_REPOSITORY || 'local/repo',
    branch: metadata.GITHUB_REF_NAME || 'local',
    commit_sha: metadata.GITHUB_SHA || 'local',
    run_id: metadata.GITHUB_RUN_ID || 'local',
    timestamp: new Date().toISOString(),
    summary: { total: tests.length, passed, failed, skipped, overall_status: failed === 0 && passed > 0 ?
'PASSED' : 'FAILED' },
    tests,
  };
}

if (process.argv[1] && path.resolve(process.argv[1]) === fileURLToPath(import.meta.url)) {
  let output = '';
  try { output = await readFile(process.env.TEST_OUTPUT || 'test-output.txt', 'utf8'); }
  catch { output = ''; }
  const report = buildReport(output);
  await writeFile(process.env.REPORT_FILE || 'test-report.json', `${JSON.stringify(report, null, 2)}\n`);
  console.log(JSON.stringify(report.summary));
}
```

`scripts/send-report.js` posts the report to the receiver. It retries three times and returns an error (and writes `report-delivery-error.txt`) if the receiver cannot be reached.

scripts/send-report.js

```
import { readFile, writeFile } from 'node:fs/promises';

const reportPath = process.env.REPORT_FILE || 'test-report.json';
const report = JSON.parse(await readFile(reportPath, 'utf8'));
console.log(JSON.stringify(report.summary));
```

```
if (!Array.isArray(report.tests) || report.tests.length === 0 || !report.summary || report.summary.total <
1) {
  console.error('The report has no test results. Run `npm test > test-output.txt` and `npm run build-
report` first.');
  process.exit(1);
}

const url = process.env.RECEIVER_URL;
if (!url) {
  console.log('RECEIVER_URL is not set; report is available as an Actions artifact only.');
  process.exit(0);
}
try {
  if (!new URL(url).pathname.endsWith('/report')) throw new Error('RECEIVER_URL must end in /report.');
} catch (error) {
  console.error(`Invalid RECEIVER_URL: ${error.message}`);
  process.exit(1);
}

const headers = {
  'content-type': 'application/json',
  'ngrok-skip-browser-warning': 'true',
};
if (process.env.REPORT_TOKEN) headers['x-report-token'] = process.env.REPORT_TOKEN;

let lastError;
for (let attempt = 1; attempt <= 3; attempt += 1) {
  try {
    const response = await fetch(url, { method: 'POST', headers, body: JSON.stringify(report), signal:
AbortSignal.timeout(20_000) });
    const body = await response.text();
    if (!response.ok) throw new Error(`Receiver returned ${response.status}: ${body}`);
    console.log(`Receiver replied: ${response.status} ${body}`);
    process.exit(0);
  } catch (error) {
    lastError = error;
    console.error(`Attempt ${attempt} failed: ${error.message}`);
    if (/Receiver returned (400|401|403|404):/.test(error.message)) {
      if (error.message.includes('ERR_NGROK_3200') || error.message.includes('endpoint') &&
error.message.includes('offline')) {
        console.error('ngrok is offline. Start the receiver with `npm run receiver`, start `ngrok http
5050`, then update RECEIVER_URL to the current HTTPS URL ending in /report.');
      } else if (error.message.includes('404')) {
        console.error('Check that RECEIVER_URL uses the current ngrok HTTPS address and ends in
/report.');
      }
      break;
    }
    if (attempt < 3) await new Promise((resolve) => setTimeout(resolve, 3000));
  }
}
await writeFile('report-delivery-error.txt', `${lastError?.stack || lastError}\n`);
process.exitCode = 1;
```

5.3 Start the receiver

1. Start the receiver with the secret token and open http://localhost:5050

Terminal

```
REPORT_TOKEN="devops2026" npm run receiver

[receiver] listening on http://localhost:5050/report
[receiver] saved .../HaroonIjaz(FA23-BSE-002)-DEVOPS-LAB-MID/test-report.json: commit=local status=PASSED
passed=5/5
```

Figure 12: *Receiver running in the terminal*

5.4 Start the tunnel

Terminal

```
ngrok http 5050
```

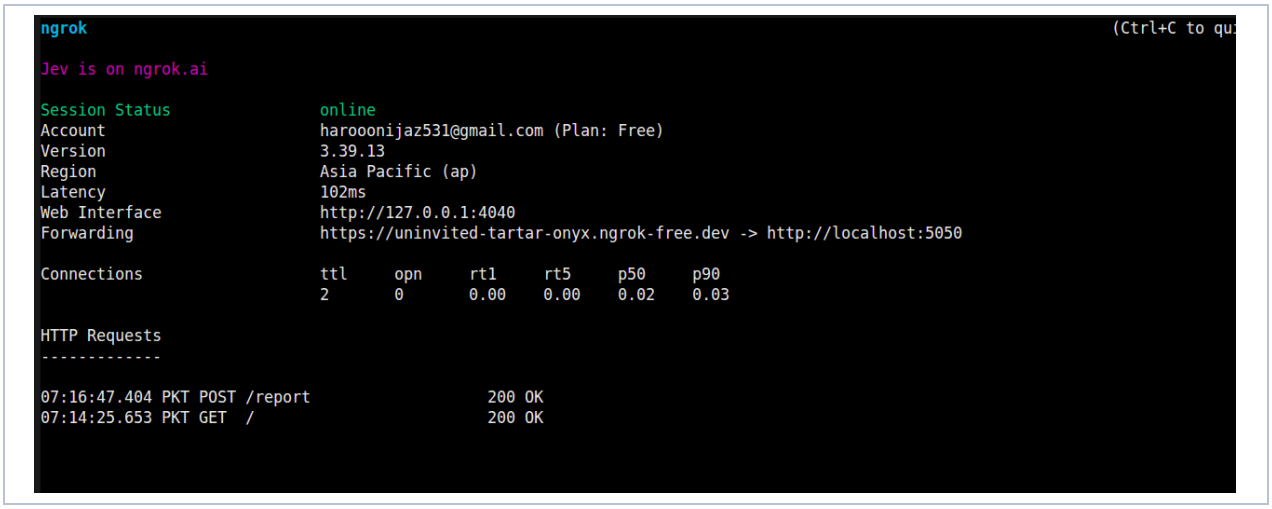


Figure 13: *ngrok showing the forwarding URL*

5.5 Add the repository secrets

Table 4: *Repository secrets for report delivery*

Secret name	Value
RECEIVER_URL	https://uninvited-tartar-onyx.ngrok-free.dev/report
REPORT_TOKEN	devops2026



Figure 14: *GitHub Actions secrets page showing RECEIVER_URL and REPORT_TOKEN*

5.6 Add the report job to the pipeline

The `report` job runs after the `test` job, even when the tests fail (`if: always()`), so failing runs are also reported.

.github/workflows/ci.yml (report job)

```
  report:
    needs: test
    if: always()
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - uses: actions/download-artifact@v4
        with:
          name: ci-test-output
      - name: Build detailed test report
        run: npm run build-report
      - name: Send report to ngrok receiver
        run: npm run send-report
        env:
          RECEIVER_URL: ${{ secrets.RECEIVER_URL }}
          REPORT_TOKEN: ${{ secrets.REPORT_TOKEN }}
      - name: Email the report (bonus)
        if: always() && hashFiles('test-report.json') != ''
        run: npm run send-email
        env:
          SMTP_USERNAME: ${{ secrets.SMTP_USERNAME }}
          SMTP_PASSWORD: ${{ secrets.SMTP_PASSWORD }}
          EMAIL_TO: ${{ secrets.EMAIL_TO || 'haroonijaz123@gmail.com' }}
      - name: Keep the report as a build artifact
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-report
          path: test-report.json
```

5.7 Push and verify

Terminal (send-report.js output)

```
{"total":5,"passed":5,"failed":0,"skipped":0,"overall_status":"PASSED"}
Receiver replied: 200 {"message":"Report saved"}
```

Figure 15: *Receiver replied "Report saved" and printed the saved message after the push*

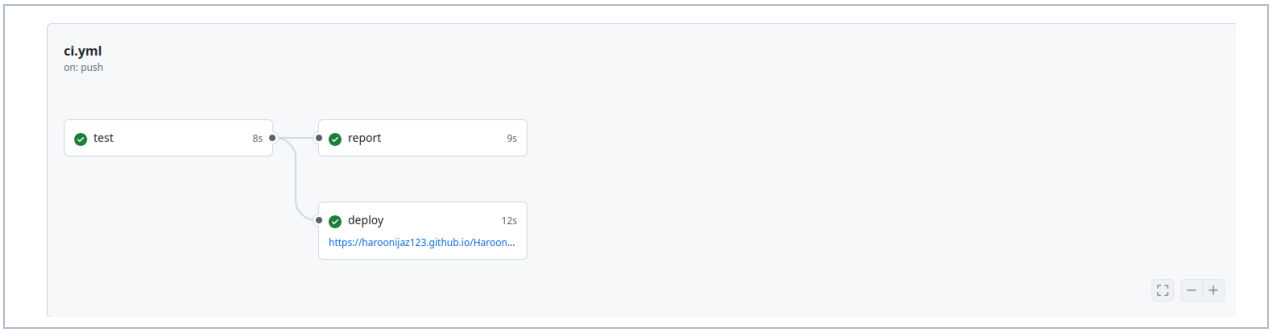


Figure 16: *Actions run showing the test, report and deploy jobs*

test-report.json

```
{
  "repository": "HaroonIjaz123/HaroonIjaz-FA23-BSE-002-DEVOPS-LAB-MID",
  "branch": "main",
  "commit_sha": "local",
  "run_id": "local",
  "timestamp": "2026-10-31T01:29:54.177Z",
  "summary": {
    "total": 5,
    "passed": 5,
    "failed": 0,
    "skipped": 0,
    "overall_status": "PASSED"
  },
  "tests": [
    {
      "name": "valid submission is saved",
      "result": "passed",
      "failure_message": null
```

```
    },
    {
      "name": "missing registration number is rejected and not saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "invalid URL is rejected and not saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "valid URLs are accepted and bad URLs are rejected",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "CI report builder keeps individual pass, fail, and skipped test results",
      "result": "passed",
      "failure_message": null
    }
  ]
}
```


Figure 17: *test-report.json created by the pipeline for a passing run*

Part 6 – Demonstrate the Complete Workflow

Table 5: Three pipeline runs used for the demonstration

Run	Change pushed	Pipeline result	test-report.json
1	Small harmless change	Green, all 5 tests pass	overall_status = PASSED, failed = 0
2	`Defect in isValidUrl`	Red, test stage fails	overall_status = FAILED, failure message present
3	Defect fixed	Green again	overall_status = PASSED

6.1 Run 1 – passing run

A small harmless change (an empty line) is committed and pushed.

Terminal

```
git add .
git commit -m "Run 1: added empty line in validation.js"
git push
```

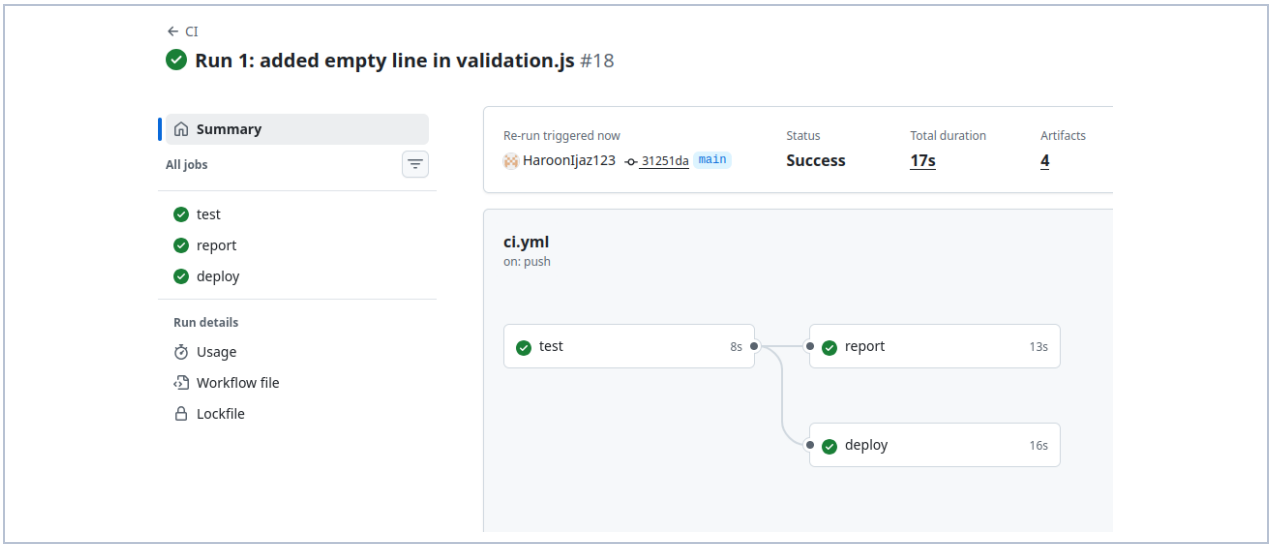


Figure 18: Run 1: Actions run with all jobs green

6.2 Run 2 – introduce a defect and fail

In `src/validation.js, isValidUrl()` changed so that every URL is accepted: the format check at the start is removed and the function returns `true` for anything that `new URL()` can parse.

src/validation.js (defect)

```
export function isValidUrl(value) {
  const trimmed = (value ?? '').trim();
  if (!trimmed) return false;          // format check removed

  try {
    const parsed = new URL(trimmed);
    return true;                       // defect: every parsable URL is accepted
  } catch {
    return false;
```

```
  }
}
```

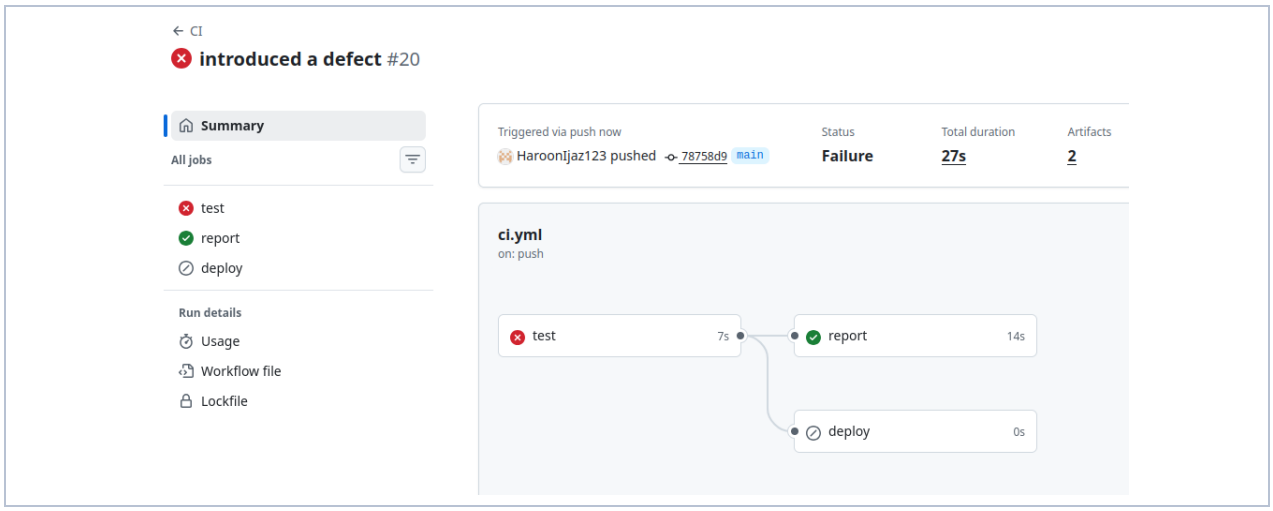


Figure 19: *Run 2: Actions run with the test job failed (red)*

Terminal (npm test with the defect)

```
> node --test --test-reporter=spec test/server.test.js
✓ valid submission is saved (7.716184ms)
✓ missing registration number is rejected and not saved (2.345957ms)
✓ invalid URL is rejected and not saved (2.067546ms)
✖ valid URLs are accepted and bad URLs are rejected (0.924683ms)
✓ CI report builder keeps individual pass, fail, and skipped test results (1.128562ms)
ℹ tests 5
ℹ suites 0
ℹ pass 4
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 123.255805
✖ failing tests:
test at test/server.test.js:40:1
✖ valid URLs are accepted and bad URLs are rejected (0.924683ms)
  AssertionError [ERR_ASSERTION]: ftp://example.com/file
  true !== false
    generatedMessage: false,
    code: 'ERR_ASSERTION',
    actual: true,
    expected: false,
    operator: 'strictEqual',
    diff: 'simple'
  }
```

Figure 20: *Run 2: failed test details in the test log*

test-report.json


```
{
  "repository": "HaroonIjaz123/HaroonIjaz-FA23-BSE-002-DEVOPS-LAB-MID",
  "branch": "main",
  "commit_sha": "local",
  "run_id": "local",
  "timestamp": "2026-10-31T01:29:54.141Z",
  "summary": {
    "total": 7,
    "passed": 4,
    "failed": 3,
```

```
    "skipped": 0,
    "overall_status": "FAILED"
  },
  "tests": [
    {
      "name": "valid submission is saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "missing registration number is rejected and not saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "invalid URL is rejected and not saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "valid URLs are accepted and bad URLs are rejected",
      "result": "failed",
      "failure_message": "See the test job log for details."
    },
    {
      "name": "CI report builder keeps individual pass, fail, and skipped test results",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "failing tests:",
      "result": "failed",
      "failure_message": "See the test job log for details."
    },
    {
      "name": "valid URLs are accepted and bad URLs are rejected",
      "result": "failed",
      "failure_message": "See the test job log for details."
    }
  ]
}
```

Figure 21: *Run 2: test-report.json with overall_status FAILED and the failure_message*

6.3 Run 3 – fix the defect

src/validation.js (fix)

```
export function isValidUrl(value) {
  const trimmed = (value ?? '').trim();
  if (!trimmed || /\s/.test(trimmed) || !/^https?:\/\/[^/?#]+/i.test(trimmed)) return false;

  try {
    const parsed = new URL(trimmed);
    return ['http:', 'https:'].includes(parsed.protocol) && Boolean(parsed.host);
  } catch {
    return false;
  }
}

export function validateSubmission(form) {
  const errors = [];
  if (!(form.name ?? '').trim()) errors.push('Name is required.');
  if (!(form.reg_no ?? '').trim()) errors.push('Registration number is required.');
  if (!(form.title ?? '').trim()) errors.push('Assignment title is required.');
  const url = (form.url ?? '').trim();
  if (!url) errors.push('Submission URL is required.');
  else if (!isValidUrl(url)) errors.push('Submission URL is not a valid http(s) URL.');
  return errors;
}
```

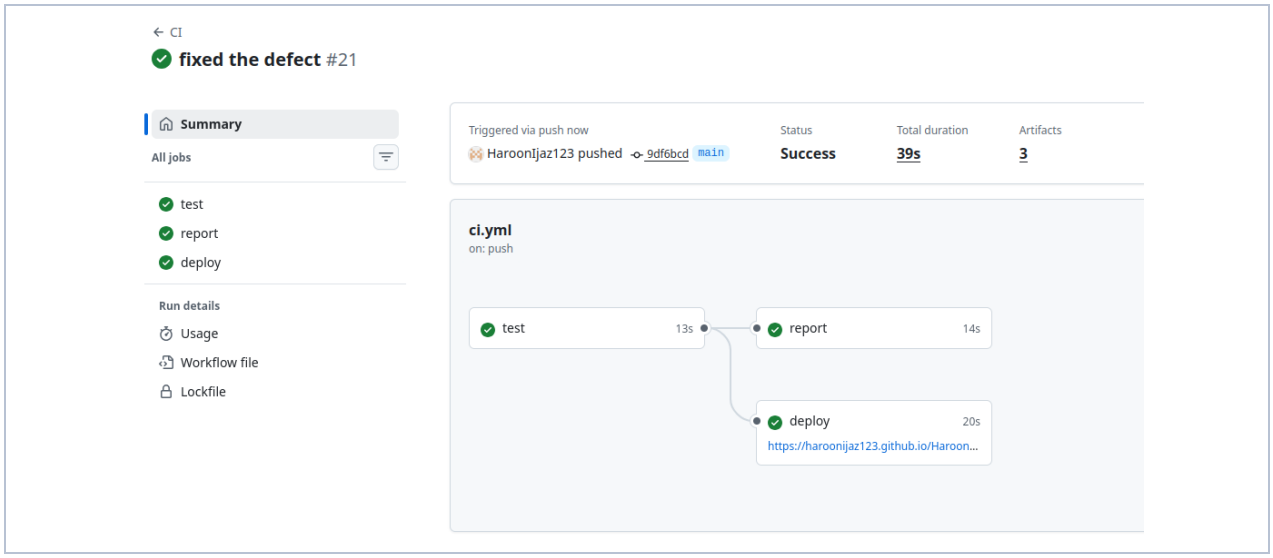


Figure 22: *Run 3: Actions run green again*

test-report.json


```
{
  "repository": "HaroonIjaz123/HaroonIjaz-FA23-BSE-002-DEVOPS-LAB-MID",
  "branch": "main",
  "commit_sha": "local",
  "run_id": "local",
  "timestamp": "2026-10-31T01:29:54.177Z",
  "summary": {
    "total": 5,
    "passed": 5,
    "failed": 0,
    "skipped": 0,
    "overall_status": "PASSED"
  },
  "tests": [
    {
      "name": "valid submission is saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "missing registration number is rejected and not saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "invalid URL is rejected and not saved",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "valid URLs are accepted and bad URLs are rejected",
      "result": "passed",
      "failure_message": null
    },
    {
      "name": "CI report builder keeps individual pass, fail, and skipped test results",
      "result": "passed",
      "failure_message": null
    }
  ]
}
```


Figure 23: *Run 3: test-report.json with overall_status PASSED*

Part 7 – Bonus Tasks

7.1 Conditional deployment to GitHub Pages

The Pages job builds the React application with `npm run build` and publishes the `dist` folder. Because `vite.config.js` sets `base: './'`, the built files work from the repository sub-path on GitHub Pages. GitHub Pages has no Node server, so the page automatically stores submissions in the visitor's browser.

dist/index.html (generated by npm run build)

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <meta name="theme-color" content="#1f3864" />
    <title>Assignment Submission Portal</title>
    <script type="module" crossorigin src="./assets/index-C1ONHo8t.js"></script>
    <link rel="stylesheet" crossorigin href="./assets/index-BziQKNzl.css">
  </head>
  <body>
    <div id="root"></div>
  </body>
</html>
```

Enable Pages

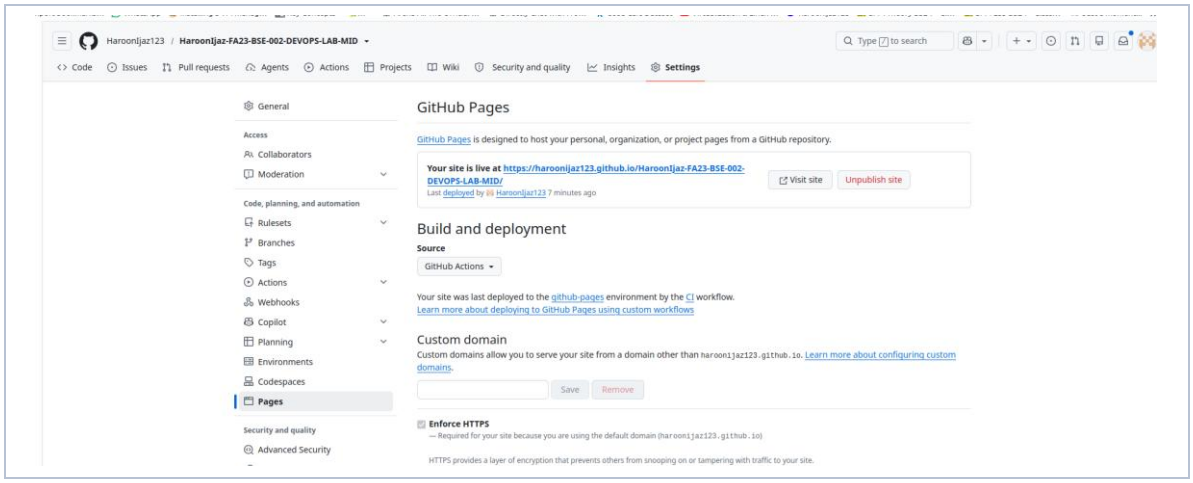


Figure 24: *Settings > Pages with the source set to GitHub Actions*

Add the deploy job

`needs: test` means the deploy job starts only when the test job succeeded. If any test fails the deploy job is skipped, so the previously published version stays unchanged. The `if` condition restricts deployment to the main branch.

.github/workflows/ci.yml (deploy job)

```
  deploy:
    needs: test                        # runs only if the test job succeeded
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    permissions:
      pages: write
```

```
      id-token: write
    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run build
      - uses: actions/configure-pages@v5
      - uses: actions/upload-pages-artifact@v3
        with:
          path: dist
      - id: deployment
        uses: actions/deploy-pages@v4
```

1. URL: https://haroonijaz123.github.io/HaroonIjaz-FA23-BSE-002-DEVOPS-LAB-MID/

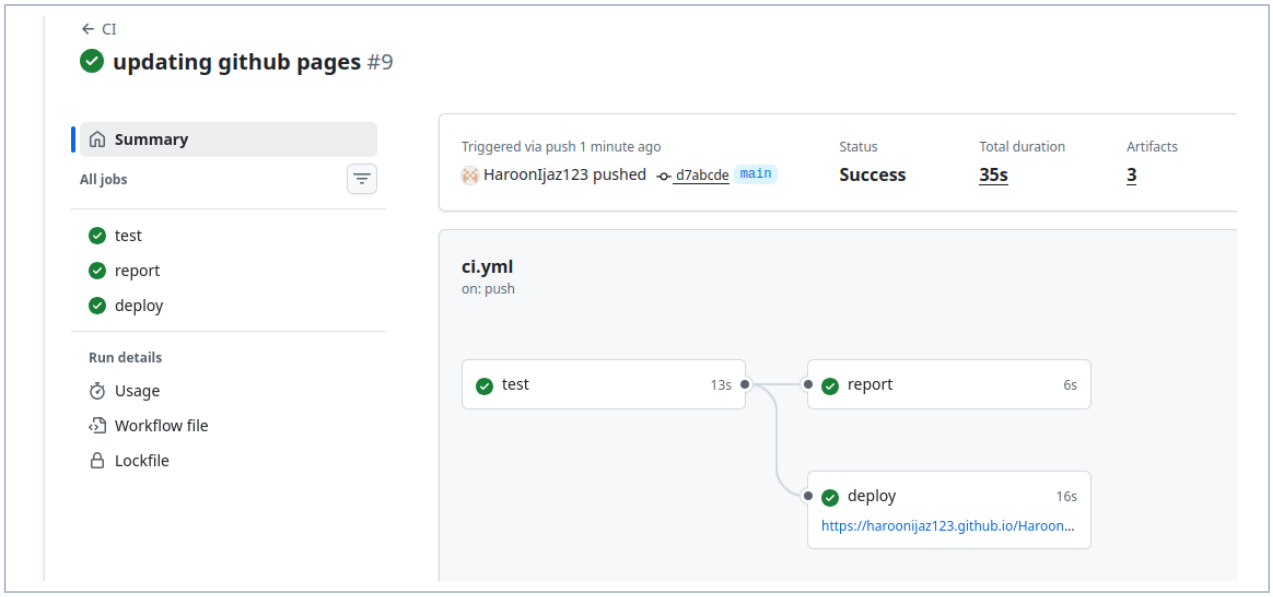


Figure 25: *Actions run with the deploy job successful*

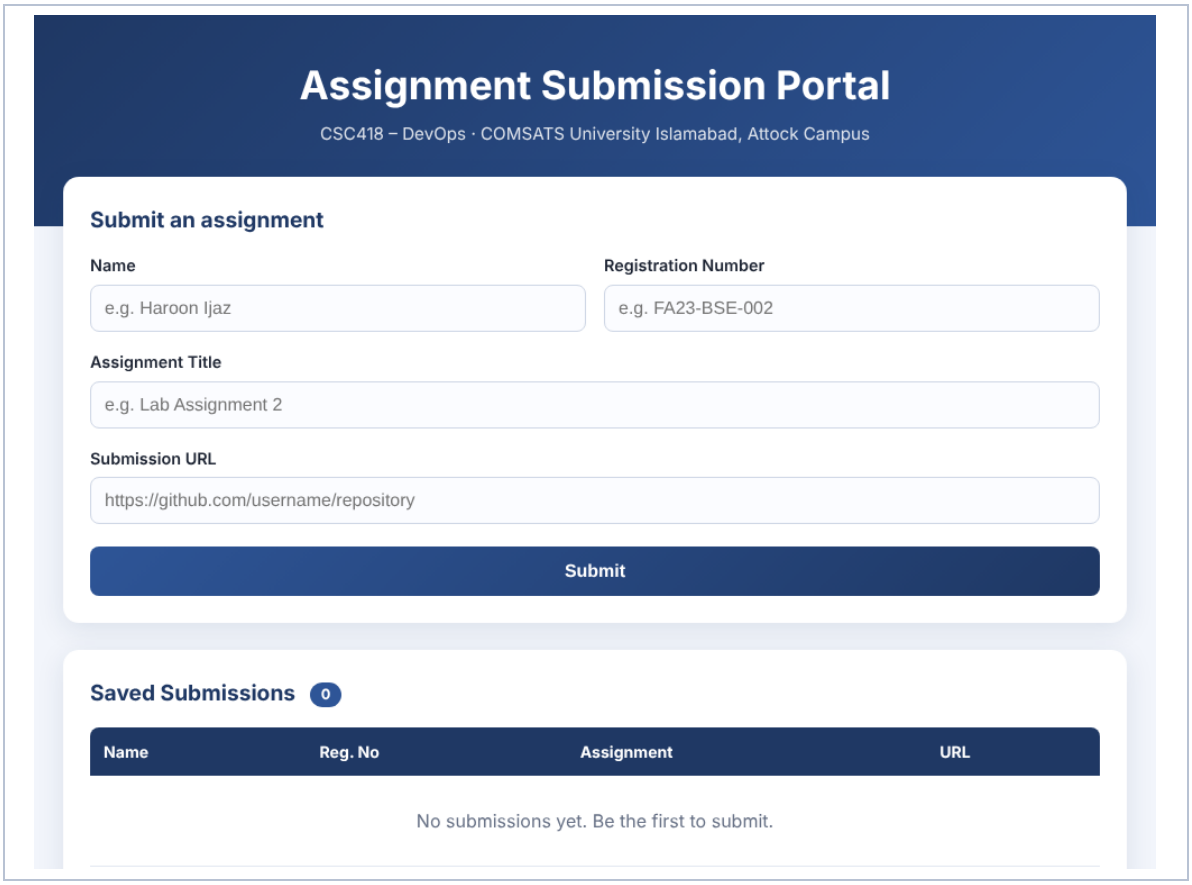


Figure 26: *Static React build (the files in dist) as published on GitHub Pages, running without the API*

7.2 Email report

`scripts/send-email.js` sends the report to **haroonijaz123@gmail.com** after both passing and failing runs. The subject shows the result, the repository and the short commit id. The message body contains the commit identifier, overall result, test counts and each individual test result, and `test-report.json` is attached. The script uses the `nodemailer` package, and the SMTP credentials are read from GitHub secrets (environment variables), never from the code.

Add secrets

Table 7: *Repository secrets for the email report*

Secret name	Value
`SMTP_USERNAME`	haroonijaz123@gmail.com (the Gmail address that sends the e-mail)
`SMTP_PASSWORD`	The app password
`EMAIL_TO`	haroonijaz123@gmail.com (recipient)

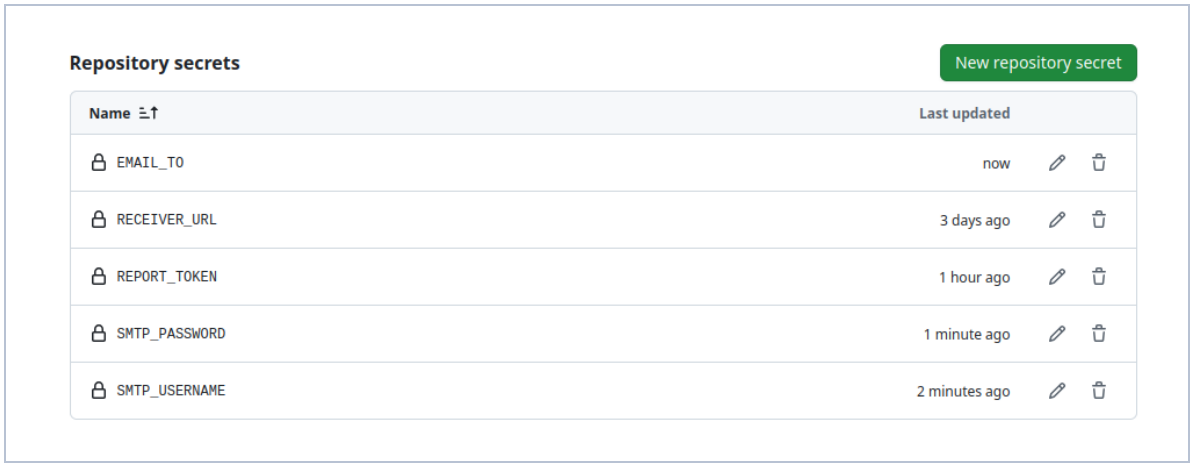


Figure 27: *Actions secrets list showing the email secrets (values hidden)*

Create scripts/send-email.js

scripts/send-email.js

```
import { readFile } from 'node:fs/promises';
import nodemailer from 'nodemailer';

const user = process.env.SMTP_USERNAME || '';
const password = process.env.SMTP_PASSWORD || '';
const recipient = process.env.EMAIL_TO || 'harooonijaz123@gmail.com';
if (!(user && password && recipient)) {
  console.log('SMTP_USERNAME / SMTP_PASSWORD / EMAIL_TO not set; email skipped.');
  process.exit(0);
}

const report = JSON.parse(await readFile(process.env.REPORT_FILE || 'test-report.json', 'utf8'));
const summary = report.summary;
const lines = [
  `Repository : ${report.repository}`,
  `Branch     : ${report.branch}`,
  `Commit     : ${report.commit_sha}`,
  `Run ID     : ${report.run_id}`,
  `Result     : ${summary.overall_status}`,
  `Total/Passed/Failed : ${summary.total}/${summary.passed}/${summary.failed}`,
  '',
  ...report.tests.map((item) => `- ${item.name}: ${item.result}${item.failure_message ? `
(${item.failure_message.slice(0, 200)})` : ''}`),
];
const port = Number(process.env.SMTP_PORT || 465);
const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST || 'smtp.gmail.com',
  port,
  secure: port === 465,
  auth: { user, pass: password },
  connectionTimeout: 30_000,
  greetingTimeout: 30_000,
  socketTimeout: 30_000,
});
await transporter.sendMail({
  from: user,
  to: recipient,
  subject: `[CI ${summary.overall_status}] ${report.repository} @ ${report.commit_sha.slice(0, 7)}
(${summary.passed}/${summary.total} passed)`,
  text: lines.join('\n'),
  attachments: [{ filename: 'test-report.json', content: JSON.stringify(report, null, 2), contentType:
'application/json' }],
});
console.log(`Report emailed to ${recipient}`);
```

Add the email step

.github/workflows/ci.yml (email step inside the report job)

```
        REPORT_TOKEN: ${{ secrets.REPORT_TOKEN }}
    - name: Email the report (bonus)
      if: always() && hashFiles('test-report.json') != ''
      run: npm run send-email
      env:
        SMTP_USERNAME: ${{ secrets.SMTP_USERNAME }}
        SMTP_PASSWORD: ${{ secrets.SMTP_PASSWORD }}
        EMAIL_TO: ${{ secrets.EMAIL_TO || 'haroonijaz123@gmail.com' }}
    - name: Keep the report as a build artifact
```

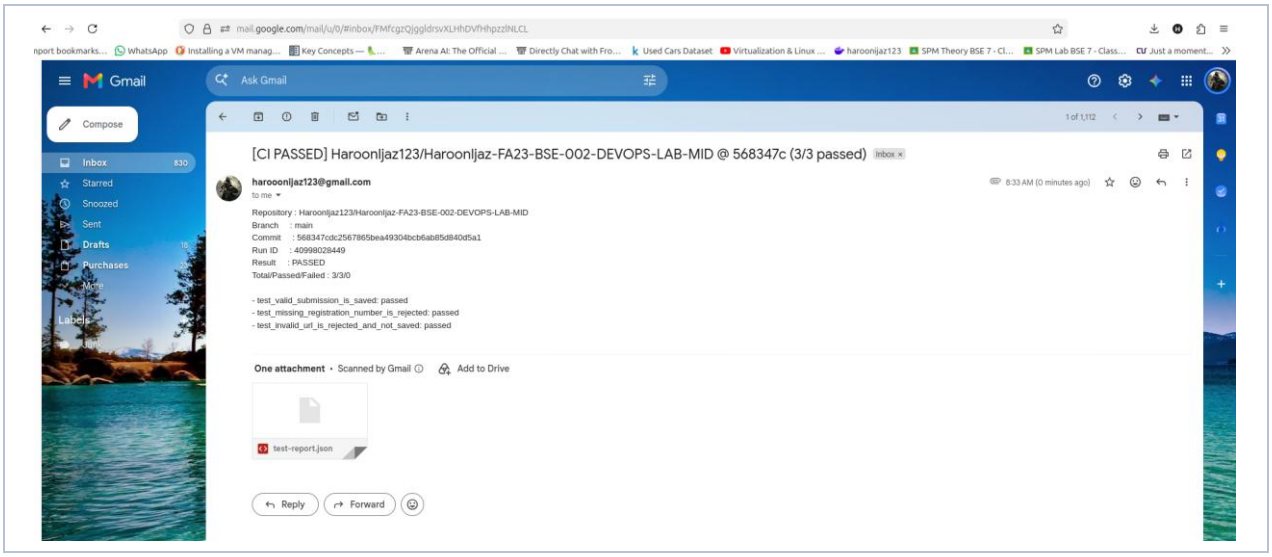


Figure 28: *Received e-mail for a passing run (with attached test-report.json)*